



UNIVERSITATEA „ALEXANDRU IOAN CUZA” DE IASI

COURSE PROGRAMME

1. Information about the programme

1.1 University	University "Alexandru Ioan Cuza" of Iasi
1.2 Faculty	Faculty of Computer Science
1.3 Department	Department of Computer Science
1.4 Domain	Computer Science
1.5 Cycle	Masters
1.6 Programme / Qualification	Studii avansate în informatică/ Advanced Studies in Computer Science

2. Information about the course

2.1 Course Name	Formal Methods in Software Engineering						
2.2 Course taught by	Assoc. Prof. PhD. ANDREI ARUSOAIE						
2.3 Seminary / laboratory taught by	Assoc. Prof. PhD. ANDREI ARUSOAIE						
2.4 Year	I	2.5 Semester	I	2.6 Type of evaluation*	E	2.7 Course type**	Op

*E – Exam / C – Colloquium / V – Verification

**OB – Obligatory / OP – Optionally / F – Facultative

3. Total hours (estimated per semester and activities)

3.1 Number of hours per week	4	3.2 course	2	3.3 seminary/laboratory	2
3.4 Total number of hours	56	3.5 course	28	3.6 seminary/laboratory	28
Distribution					hours
Individual study using textbooks, course notes, bibliography items, etc.					28
Supplementary study (library, on-line platforms, etc.)					11
Individual study for seminary/laboratory, homeworks, projects, etc.					30
Tutoring					0
Examination					0
Other activities					0

3.7 Total hours of individual activity*	69
3.8 Total hours per semester	125

3.9 Credit points	5
-------------------	---

4. Pre-requisites - Curriculum (if necessary)

Data Structures & Algorithm Design, Software Engineering, Logic in Computer Science

5. Conditions (if necessary)

5.1 Course	The ability to design an algorithm. The ability to work with basic concepts from mathematical logic, sets, functions, and relations.
5.2 Seminary / Laboratory	The ability to implement a non-trivial algorithm. Experience with at least one common programming language (Java, C, C++, Python).

6. Objectives

Upon successful completion of this discipline, students will be able to:

1. Formalise the preconditions and postconditions of a function/procedure;
2. Formally model properties of algorithms;
3. Use tools for the automatic verification of software properties;
4. Use tools for the analysing software systems.

7. Specific competencies/Learning outcomes

- The student describes advanced software engineering concepts, development methodologies, DevOps practices, and core software engineering fundamentals. The student explains software testing and quality assurance methods and describes specialization options in artificial intelligence and distributed systems applied to software. The student describes concepts and technologies for autonomous systems, web applications, advanced analytics, and automated reasoning relevant to intelligent software systems. The student explains complementary topics related to computer vision, neural networks, blockchain, and emerging technologies relevant to intelligent software systems.
- The student will be able to apply the acquired knowledge to analyze and model real problems in classical cryptography and security, proposing rigorously substantiated solutions with demonstrations of correctness and performance. The student will be able to apply the acquired knowledge to analyze and model real-world cryptographic and post-quantum security problems, proposing rigorously substantiated solutions with demonstrations of correctness and performance. The student will be able to apply the acquired knowledge to analyze and model real problems in advanced cryptography with applications in cloud security, proposing rigorously substantiated solutions with demonstrations of correctness and performance.
- The student will use the acquired knowledge responsibly, making informed decisions based on the knowledge acquired and extended autonomously.

8. Contents

8.1 Course	Teaching methods	Remarks (number of hours, references)
1. Introduction in Formal Methods. Motivation.	lecture	2 hours; slides and/or lecture notes
2. Formal Systems, Inference Rules, and Proofs	lecture; live coding sessions	2 hours; slides and/or lecture notes
3. Syntax and Operational Semantics	lecture; live coding sessions	2 hours; slides and/or lecture notes

8.1 Course	Teaching methods	Remarks (number of hours, references)
4. Hoare Logic & Weakest Precondition Calculus	lecture; live coding sessions	2 hours; slides and/or lecture notes
5. The Weakest Precondition Calculus	lecture; live coding sessions	2 hours; slides and/or lecture notes
6. The Strongest Postcondition Calculus	lecture; live coding sessions	2 hours; slides and/or lecture notes
7. More on Proofs: a verified quicksort	lecture, discussion, case study	2 hours; lecture notes
8. Evaluation: Case Study	discussions	2 hours
9. Interactive Theorem Proving in Lean 4	lecture; live coding sessions	2 hours; slides and/or lecture notes
10. Writing Proofs in Lean 4	lecture; live coding sessions	2 hours; slides and/or lecture notes
11. Induction and Other Proof Techniques	lecture; live coding sessions	2 hours; slides and/or lecture notes
12. Certified Compilation	lecture; live coding sessions	2 hours; slides and/or lecture notes
13. Type Systems	lecture; live coding sessions	2 hours; slides and/or lecture notes
14. Introduction to Static Analysis	lecture	2 hours; slides and/or lecture notes

Bibliography

- [1] Michael Huth, Mark Ryan. Logic In Computer Science. Modelling and Reasoning about Systems.
- [2] Boro Sitnikovski, Introducing Software Verification with Dafny Language – Proving Program Correctness, ISBN, 978-1-4842-7977-9.
- [3] Flemming Nielson, Hanne R. Nielson, Chris Hankin: Principles of Program Analysis. 2nd edition, Springer, 2005
- [4] University of Pennsylvania, CIS 700: Software Analysis and Testing (Fall 2018) [link](#)
- [5] Manuel Clavel, Francisco Durán, Steven Eker, Santiago Escobar, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, Rubén Rubio, Carolyn Talcott, Maude Manual, 2024 [link](#)
- [6] Benjamin C. Pierce, Types and Programming Languages, MIT Press, 2002.
- [7] Leslie Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers, ISBN-13 978-0321143068, 2002
- [8] Glynn Winskel, The formal semantics of programming languages - an introduction. MIT Press 1993, ISBN 978-0-262-23169-5.

8.2 Seminary / Laboratory	Teaching methods	Remarks (number of hours, references)
1. Empirical bug detection in programs.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
2. Dafny introduction: small programs and very basic proofs.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
3. A small programming language interpreter based on formal semantics. Basic calculational proof.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
4. Pre/post-conditions. Invariants.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below

8.2 Seminary / Laboratory	Teaching methods	Remarks (number of hours, references)
5. From informal specifications of an algorithm to formal specification in Dafny.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
6. Automated verification of algorithms: case study.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
7. Advanced program verification: case study.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
8. Midterm	discussions	homework report and implementation presentation
9. The Lean proof assistant - intro. Simple inductive definitions, functions, and basic tactics.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
10. Interactive Proofs: by induction, by case analysis, and other advanced tactics.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
11. Advanced Proofs in Lean.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
12. Proof assistants at work: a certified compiler.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
13. Type Systems and their properties: practical implementation and correctness proofs.	laboratory worksheet	2 hours; lecture notes and the indicated bibliography below
14. Final evaluation project: full verification of an algorithm/protocol/mathematical property	discussions	final project report and implementation presentation

Bibliography

1. Dafny official docs: <https://dafny.org/>
2. Lean official docs: <https://lean-lang.org/learn/>

9. Coordination of the contents with the expectations of the community representatives, professional associations and relevant employers in the corresponding domain

--

10. Assessment and examination

10.1 Continuous assessment		Percentage (min. 30%)	100
Course	Assessment type		Practical assessment
	Percentage		30
	Failure to pass the continuous assessment results in failure to pass the final assessment		No

	Components / examinations	a) homework: 30 p
Seminary / Laboratory	Assessment type	Mixed assessment
	Percentage	70
	Failure to pass the continuous assessment results in failure to pass the final assessment	No
	Components / examinations	a) laboratory worksheets: 25p b) final project: 45p

10.2 Final assessment	Percentage (max. 70%)	0
	Assessment type	

10.3 Special notes (special situations is assessment)	
No reexamination.	

10.4 Minimum performance standard	
Obtain at least 50% from the maximum of 100 points.	

Date,
Course coordinator,
Assoc. Prof. PhD. ANDREI ARUSOAIE

Seminary coordinator,
Assoc. Prof. PhD. ANDREI ARUSOAIE

Approval date in the department,

Head of the departament,
Assoc. Prof. PhD. ANDREI ARUSOAIE